

IMC Generation



Date: _____

Day: M T W T F S

Introduction :-

- o Intermediate code generation is the fourth phase of compiler design.
- o The task of compiler is to convert the source code into machine program.
- o This activity can be done directly, but it is not always possible to generate such a machine code directly in one pass.
- o Then typically compilers generate an easy to represent form of source language which is called intermediate language.
- o Ideally, details of the source language are confined to the front end, and details of the target machine to the back end.
- o This phase generate a code that is generalized code.

Date: _____

Day: MTWTFSS

- o The generation of an intermediate language leads to efficient code generation.

Properties of intermediate language:

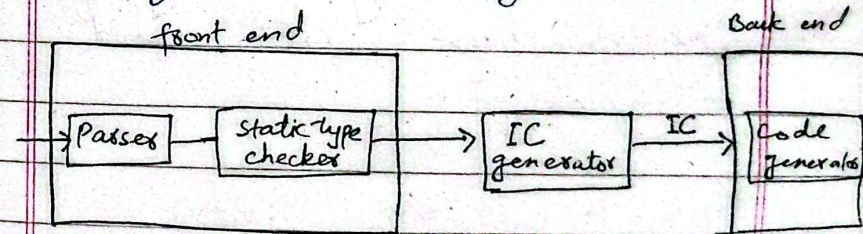
- (1) The intermediate language is an easy form of source language which can be generated efficiently by the compiler.
- (2) The generation of intermediate language should lead to efficient code generation.
- (3) The intermediate code should act as effective mediator between front and back end.
- (4) The intermediate language should be flexible enough / so that optimized code can be generated.

Date: _____

Day: MTWTFSS

Benefits of IC:

- (1) A compiler for different machines can be created by attaching different backend to the existing front ends of each machine.
- (2) A compiler for different source languages: even with same IC (1) machine) can be created by providing different front ends for corresponding source languages to existing back end.
- (3) A machine independent code optimizer can be applied to intermediate code in order to optimize the code generation.



Date: _____

- o The generation of an intermediate language leads to efficient code generation.

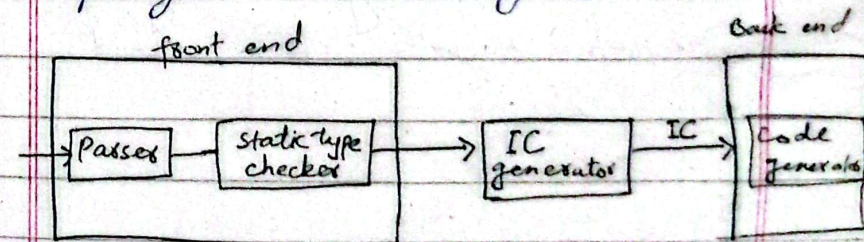
Properties of intermediate language:

- (1) The intermediate language is an easy form of source language which can be generated efficiently by the compiler.
- (2) The generation of intermediate language should lead to efficient code generation.
- (3) The intermediate code should act as effective mediator between front and back end.
- (4) The intermediate language should be flexible enough / so that optimized code can be generated.

Date: _____

Benefits of ICL: of chaitin

- (1) A compiler for different machines can be created by attaching different backend to the existing front ends of each machine.
- (2) A compiler for different source languages: even with different (1) machine) can be created by providing different front ends for corresponding source languages to existing back end.
- (3) A machine independent code optimizer can be applied to intermediate code in order to optimize the code generation.



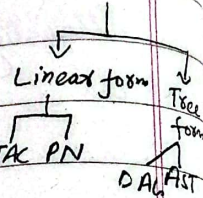
Date: _____

Day: MTWTF

Methods to Represent Code

⇒ There are mainly three types of intermediate code representation:

1. Abstract syntax tree
2. Polish Notation
3. Three address code

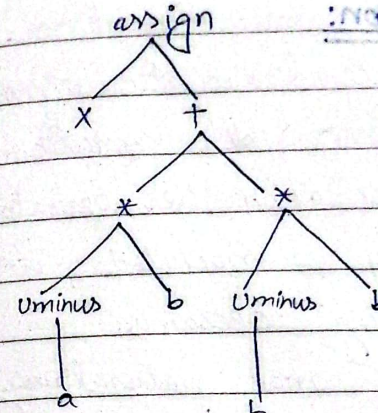


(1) Abstract Syntax tree:

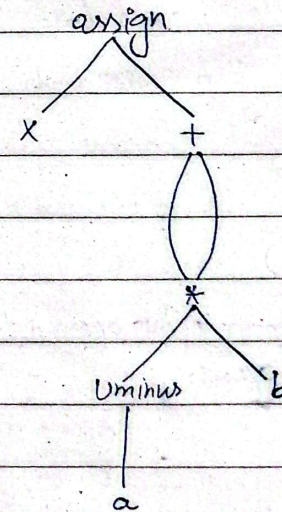
o The natural hierarchical structure is represented by syntax trees. DAG is very much similar to syntax trees but they are in more compact form.

o The code being generated as intermediate should be such that the remaining processing of the subsequent phases should be easy.

Date: _____

Day: MTWTF

Syntax tree i/p: $x = -a * b + -a * b$



DAG i/p: $x = -a * b + -a * b$

Date: _____

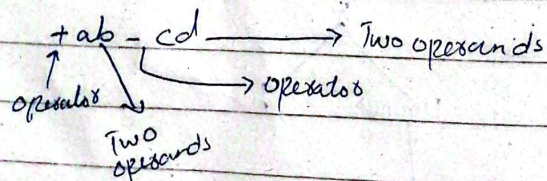
Day: MTWTF

2) Polish notation:

- Basically, the linearization of syntax trees is polish notation.
- In this representation, the operators can be easily associated with the corresponding operands.
- This is the most natural way of representation in expression evaluation.
- The polish notation is also called as prefix notation in which the operator occurs first and then operands are arranged.

for example:

$$(a+b) * (c-d)$$

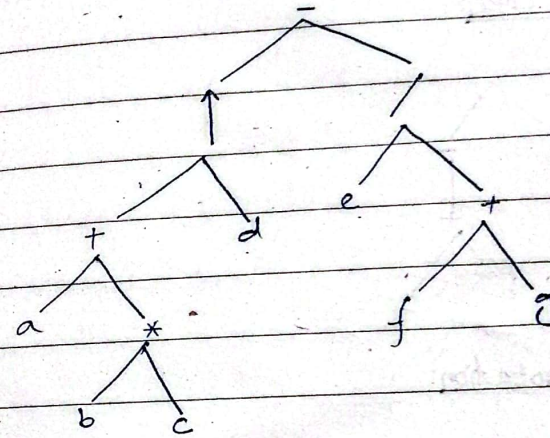


- There is a reverse polish notation which is used using postfix or postfix representation.

Date: _____

Day: MTWTF

Q. Construct syntax tree and postfix notation for the following expression:
 $(a+(b*c)) \uparrow d-e / (f+g)$



Postfix expression:

$$(a+(b*c)) \uparrow d-e / (f+g)$$

$$(a+T_1) \uparrow d-e / (f+g) \quad \text{where } T_1 = *bc*$$

$$T_2 \uparrow d-e / (f+g) \quad \text{where } T_2 = aT_1 +$$

$$T_3 = e / (f+g) \quad \text{where } T_3 = T_2 \uparrow$$

$$T_4 = e / T_3 \quad \text{where } T_4 = f+g$$

$$T_5 = T_3 \quad \text{where } T_5 = eT_4 /$$

$$T_6 = T_5 T_2$$

Date: _____

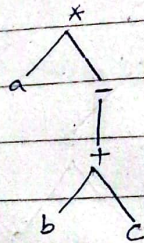
Day: (M T W T F S)

Q. Translate the arithmetic expression $a * (b + c)$ into:

(1) Syntax tree

(2) Post-fix notation

(1) Syntax tree:



(2) Post-fix notation:

$a * T_1$ where $T_1 = b c +$
 T_2 where $T_2 = a T_1 *$

Substituting in backward direction:

$a T_1 *$

$a b c + *$ is postfix.

Date: _____

Day: (M T W T F S)

Q. Three address code:

- o In three address code form at the most three addresses are used to represent any statement. it is an intermediate code.
- o The general form of three address code representation is:

$a := b \text{ op } c$

- o Where a, b or c are the operands that can be names, constants, compiler generated temporaries and op represents the operators.
- o The operators can be fixed or floating point arithmetic operators or logical operators on Boolean valued data.
- o Only single operation at right side of the expression is allowed at a time.

Date: _____

Day: MTWTF

⇒ For the expression $a = b + c + d$ like assembly code will be:

$t1 = b + c$

$t2 = t1 + d$

$a = t2$

⇒ Here $t1$ and $t2$ are the temporary names generated by the compiler.

⇒ There are at the most three addresses allowed (two for operands and one for result).

Hence, the name of this representation is three-address code.

⇒ Three address code is used by the optimizing compiler.

⇒ it has at most three operands.

⇒ it is a combination of assignment and a binary operator.

Date: _____

Day: MTWTF

Various statements in three-address code:

(1) Assignment statement:

$x = y \text{ op } z$

$x = \text{op } y$

$x = y$

(2) Jump:

Conditional:

if $x \text{ rel op } y$ goto L

Unconditional:

goto L

(3) Array Assignment:

$x = y[i]$

$x[i] = y$

(4) Pointer address Assign:

$x = \&y$

$x = *y$

Note:

o TAC used by compiler to implement code optimization.

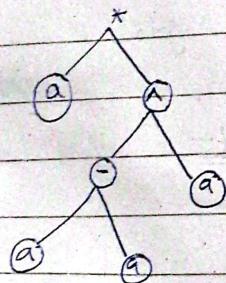
Q. Write three address code of expression $a * (b + c)$.

$t_1 := b + c$

$t_2 := -t_1$

$t_3 := a * t_2$

Q. Draw syntax tree for: $a * (a - a)$



Pros of Three address code:

- o Easy to rearrange code for global optimization.
- o Quickly access value of temporary variables using symbol table.

Cons of three address code:

- o Contain lot of temporaries.
- o Temporary variable creation increases time and space complexity.

Various Representations of three address code:

⇒ Three address code is an abstract form of intermediate code that can be implemented as a record with the address field.

⇒ These are three representations used for three address code such as quadruples, triples and indexed triples.

o Quadruple

o Triple

o indexed triple

Quadruple representation:

⇒ The quadruple is the structure with at most four fields such as:

o OP o arg1 o arg2 o result

OP	arg1	arg2	result
----	------	------	--------

Date: _____

- The op field is used to represent the operator.
- The arg1 and arg2 represent the two operands used and
- result field is used to store the result of an expression.

for example:

$$x := -a \times b + -a \times b$$

Three address code is

$$t_1 := -a$$

$$t_2 := t_1 \times b$$

$$t_3 := -a$$

$$t_4 := t_3 \times b$$

$$t_5 := t_2 + t_4$$

$$x := t_5$$

#	OP	arg1	arg2	result
0	Uminus	a		t ₁
1	*	t ₁	b	t ₂
2	-	a		t ₃
3	*	t ₃	b	t ₄
4	+	t ₂	t ₄	t ₅
5	:=	t ₅		x

Date: _____

(2) Triple:

- In triple representation the use of temporary variables is avoided by representing the pointers in the symbol table.

for example:

$$x := -a \times b + -a \times b$$

OP	arg1	arg2
----	------	------

Number	OP	Arg1	Arg2
(0)	Uminus	a	
(1)	*	(0)	b
(2)	Uminus	a	
(3)	*	(2)	b
(4)	+	(1)	(3)
(5)	:=	x	(4)

(3) Indirect triples:

- In the indirect triple representation the listing of triples is been done.

Date: _____

And listing pointers are used instead of using statements.

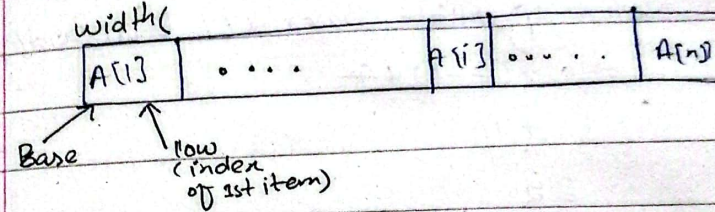
Number	Op	Arg1	Arg2
(0)	minus	a	
(1)	*	(11)	b
(2)	minus	a	
(3)	*	(13)	b
(4)	+	(12)	(14)
(5)	:=	x	(15)

Number	statement
(0)	(11)
(1)	(18)
(2)	(13)
(3)	(14)
(4)	(15)
(5)	(16)

Date: _____

Three address code (Array reference)

Three address code for 1D Array:



Base: address of starting location of array

width: width of each array element = 4 bytes

low: index of first array element

$$\text{Location of } A[i] = \text{Base} + (i - \text{low}) * \text{width}$$

low = 1
By default

$$= (\text{Base} - \text{low} * \text{width}) + i * \text{width}$$

$$= \underbrace{(\text{address}_A - \text{width})}_{t_2} + \underbrace{i * \text{width}}_{t_1}$$

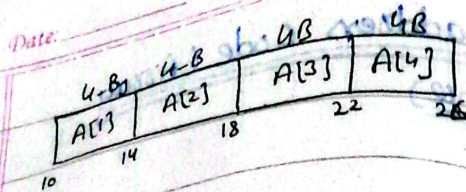
$$A[i] = t_2[t_1]$$

if $\text{low} = 0$ then

$$= \text{Base} + i * \text{width}$$

Note:

$$= A + t_1 \text{ is same as } A[t_1]$$



width = 4

Location of $A[4]$ = $\text{Base}_A + (i - \text{low}) \times \text{width}$
 $= 10 + (4 - 1) \times 4$
 $= 10 + 12$
 $= 22$

Q. Convert the following into three address code.

$a[i] = 10;$ width = 4B

$t_1 = i \times 4$ $\therefore t_1 = i \times \text{width}$
 $t_2 = \text{address}, - 4$
 $t_2[t_1] = 10$

Three - address code for

2D array:

$A[i, j] = \text{base}_A + (i - \text{low}_1) \times n_2 + j - \text{low}_2$
 $\times \text{width}$
 $= \underbrace{(i \times n_2 + j) \times \text{width}}_{t_1} + \underbrace{\text{base}_A - (\text{low}_1 \times n_2 + \text{low}_2) \times \text{width}}_{t_2}$
 $A[i, j] = t_2[t_1]$

Three - address code for

if - then - else:

Code:

if $a < b$
 then $x = y + z$
 else : $p = r + s$

Ans:

- (1) if $a < b$ goto (3) (true)
- (2) goto (6) (false)
- (3) $t_1 = y + z$
- (4) $x = t_1$
- (5) goto --
- (6) $t_2 = r + s$

Date: _____

Backpatching

Day: M T W T F S

(7) $p = t_2$

(8) goto _

3-address code for while loop:While ($a \leq b$)do $x = y + z$

- (1) if $a \leq b$ goto (3)
- (2) goto _
- (3) $t_1 = y + z$
- (4) $x = t_1$
- (5) goto (1)

3-address code for loop:

for ($i = 1$; $i \leq 20$; $i++$)
 $x = y + z$

Ans:

- (1) $i = 1$
- (2) if $i \leq 20$ goto _
- (3) goto _
- (4) $t_1 = i + 1$

Date: _____

Day: M T W T F S

- (5) $i = t_1$
- (6) goto (2)
- (7) $t_2 = y + z$
- (8) $x = t_2$
- (9) goto (4)

3-address code for Boolean Expression:

if $a \leq b$ AND $c \leq d$
 then $p = v + x$

- (1) if $a \leq b$ goto (3)
- (2) goto _
- (3) if $c \leq d$ goto (5)
- (4) goto _
- (5) $t_1 = v + x$
- (6) $p = t_1$
- (7) goto _

Q. Convert given expression into three address code.

While ($A < C$ AND $B > D$) do

if $A=1$ then $C = C+1$
else

While $A \leq D$ do

~~modi~~ $A = A+3$ ~~also modi~~

(1) if $A < C$ goto (3)

(2) goto --

(3) if $B < D$ goto (5)

(4) goto --

(5) if $A=1$ goto (7)

(6) goto (10)

(7) $t_1 = C + 1$

(8) $C = t_1$

(9) goto (1)

(10) if $A \leq D$ goto (12)

(11) goto (1)

(12) $t_2 = A + 3$

(13) $A = t_2$

(14) goto (10)

(15) goto (1)

Date: _____

Day: (M T W T F S S)

Q. Write three address code for

switch ($i+j$)
{

case 1: $(4+5) - (3+2) + (2+1)$

$x = y + 2$

case 2 :

$U = V + W$

default:

$P = x + 6$

}

Ans:

(1) $t_1 = i + j$

(2) if $t_1 = 1$ goto (5)

(3) if $t_1 = 2$ goto (8)

(4) goto (11)

(5) $t_2 = y + 2$

(6) $x = t_2$

(7) goto --

(8) $t_3 = V + W$

(9) $U = t_3$

(10) goto --

(11) $t_4 = x + 6$

(12) $P = t_4$

(13) goto --

Date: _____

Q. Write the quadruples, triple and indirect triples for the expression?

$$-(a \times b) + (c + d) - (a + b + c + d)$$

Three address code can be:

$$t_1 = a \times b$$

$$t_2 = -t_1$$

$$t_3 = c + d$$

$$t_4 = t_2 + t_3$$

$$t_5 = a + b$$

$$t_6 = t_5 + t_3$$

$$t_7 = t_4 - t_6$$

Quadruple

Location	Operator	Operand 1	Operand 2
(0)	*	a	b
(1)	Uminus	t ₁	
(2)	+	c	d
(3)	+	t ₂	t ₃
(4)	+	a	b
(5)	+	t ₅	t ₃
(6)	-	t ₄	t ₆
(7)	-		

Date: _____

Triple:

Location	Operator	OP1	OP2	Result
(1)	*	a	b	
(2)	Uminus	(1)		
(3)	+	c	d	
(4)	+	(2)	(3)	
(5)	+	a	b	
(6)	+	(5)	(3)	
(7)	-	(4)	(6)	

Indirect triple

Location	Operator	OP1	OP2
(11)	*	a	b
(12)	Uminus	(11)	
(13)	+	c	d
(14)	+	(12)	(13)
(15)	+	a	b
(16)	+	(15)	(13)
(17)	-	(14)	(16)

Date: _____

	Statement
(1)	(11)
(2)	(12)
(3)	(13)
(4)	(14)
(5)	(15)
(6)	(16)
(7)	(17)

Q. Convert the following into quadruple triple and indexed triple forms:

$$-(a+b)*(c-d)$$

The three address code is:

$$t_1 = a + b$$

$$t_2 = -t_1$$

$$t_3 = c - d$$

$$t_4 = t_2 * t_3$$

Quadruple:

Date: _____

Location	operator	Arg1	Arg2	Result
(1)	+	a	b	t ₁
(2)	Uminus	t ₁		t ₂
(3)	-	c	d	t ₃
(4)	*	t ₂	t ₃	t ₄

Triple:

Location	Operator	Arg1	Arg2
(1)	+	a	b
(2)	Uminus	(1)	
(3)	-	c	d
(4)	*	(2)	(3)

Indexed triple:

Location	Operator	Arg1	Arg2
(11)	+	a	b
(12)	Uminus	(11)	
(13)	-	c	d
(14)	*	(12)	(13)

Date: _____

	statement
(1)	(11)
(2)	(12)
(3)	(13)
(4)	(14)

Q. Write the three address code for

$x = *y;$

$a = &x;$

$t_1 = \text{ptr. of } y$

$x := t_1$

$t_2 := \text{addr. of } x;$

$a = t_2$

Q. Generate the three address code for

while ($i < 10$)

{

$x = 0;$

$i = i + 1;$

}

Date: _____

Three address code can't be generated as follows (cons. x) if

(1) if $i < 10$ goto (3)

(2) goto -- $x = x - 1$

(3) $t_1 = 0$ (goto 4) if $x < 0$

(4) $x = t_1$ $x = y$

(5) $t_2 = i + 1$

(6) $t = t_2$

(7) goto (1)

(8) Lnext

Q. write three address code

for: if $a > b$ then $x = a + b$
else

$x = a - b$

(1) if $a < b$ goto (3)

(2) goto (5)

(3) $t_1 = a + b$

(4) $x = t_1$

(5) $t_2 = a - b$ (cons. x) if $x < 0$

(6) $x = t_2$ (cons. x) if $x < 0$

(7) goto -- (cons. x) if $x < 0$

Q. Write three address code for
if ($x > 10$) then

while ($a > 10$)

$y = x + a$

else if ($y > 100$)

$y = a$

(1) if $x > 10$ goto (3)

(2) goto (8)

(3) if $a > 10$ goto (5)

(4) goto --

(5) $t1 = x + a$

(6) $y = t1$

(7) goto --

(8) if $y > 100$ goto (10)

(9) goto --

(10) $y = a$

Q. Write three address code for
while ($i < 10$)

if ($i \% 2 == 0$)

evensum = evensum + i

else

oddsum = oddsum + i

(1) if $i < 10$ goto (3)

(2) goto --

(3) if $i \% 2 == 0$ goto (5)

(4) goto (8)

(5) $t1 = \text{evensum} + i$

(6) $\text{evensum} = t1$

(7) goto (1)

(8) $t2 = \text{oddsum} + i$

(9) $\text{oddsum} = t2$

(10) goto (1)

Q. Write three address code for
if ($(a < b)$ AND ($c > d$) OR ($a > d$))
then

$z = x + y * z$

else

$z = z + 1$

Date: _____

Day: (M T W T F S S)

- (1) if $a \leq b$ goto (3)
- (2) goto (11)
- (3) if $c \leq d$ goto (7)
- (4) goto (5)
- (5) if $a > d$ goto (7)
- (6) goto (11)
- (7) $t_1 = x + y$
- (8) $t_2 = t_1 \times z$
- (9) $t_2 = t_2$
- (10) goto --
- (11) $t_3 = z + 1$
- (12) $z = t_3$

Q. Write the intermediate code

for:

While $(A > B \text{ AND } A \leq 2 \times B - 5)$

$(1) A = A + B; (2) A = A - 5$

- (1) if $A > B$ goto (3)
- (2) goto --
- (3) if $A \leq 2 \times B - 5$
- (4) $t_2 = t_1 - 5$
- (5) if $A \leq t_2$ goto (7)

Date: _____

Day: (M T W T F S S)

- (6) goto --
- (7) $t_3 = A + B$
- (8) $A = t_3$
- (9) goto (1)

Q. Write the address code

for:

while $(A < C \text{ AND } B > D)$ do

if $A = 1$ then $C = C + 1$
else

while $(A \leq D)$ do

$A = A + B$

- (1) if $A < C$ goto (3)
- (2) goto --
- (3) if $B > D$ goto (5)
- (4) goto --
- (5) if $A = 1$ goto (7)
- (6) goto (10)
- (7) $t_1 = C + 1$
- (8) $C = t_1$
- (9) goto (1)
- (10) if $A \leq D$ goto (11)

- (11) $t_2 = A + B$
 (12) $A = t_2$
 (13) goto (1)

Q. Translate the executable statements of the following C program into three-address code.

```

main()
{
    int i;
    int a[10];
    i = 1;
    while (i <= 10)
    {
        a[i] = 0;
        i = i + 1;
    }
}
  
```

Ans:

- (1) $i = 1$
 (2) if $i \leq 10$ goto (4)

- (3) goto --
 (4) $t_1 = i$
 (5) $t_2 = 4 * t_1$
 (6) $a[t_2] = 0$
 (7) $t_3 = i + 1$
 (8) $i = t_3$
 (9) goto (2)

Q. Generate the three address code for the following segment of code

```

C = 0
do
{
    if (a < b) then
        x++;
    else
        x--;
    C++;
} while (C < 5)
  
```


(2) if $a < b$ goto (4)

(3) goto (7)

(4) $t_1 = x + 1$

(5) $x = t_1$

(6) goto (10)

(7) $t_2 = x - 1$

(8) $x = t_2$

(9) goto (10)

(10) $t_3 = c + 1$

(11) $c = t_3$

(12) if $c < 5$ goto (2)

(13) goto --

Q. Write a three address code for the following statement

for ($i = 1$; $i \leq 10$; $i++$)

{

$a[i] = x * 5$;

}

The three address code is:

Date: _____

Day: M T W T F S

(1) $i = 1$

(2) $t_1 = x * 5$

$t_2 = 2a$

$t_3 = \text{sizeof}(int)$

$t_4 = t_3 * i$

$t_5 = t_2 + t_4$

$t'_5 = t_1$

$i = i + 1$

if $i \leq 10$ goto (2)

Q. Write three address code for

int i

int a[10][10]

$i = 0$

while ($i < 10$)

{

$a[i][i] = 1$;

$i++$

}

The three address code is

Date: _____

Day: MTWTFSS

Assuming 4 byte word allocation
 $a[i][j]$ will be at location

$$[address(a) + (10 \times i + j) \times 4]$$

$$\therefore a[i][j] = address(a) + 44 \times i$$

The three address code is

$$i = 0$$

L1: if $i \leq 10$ goto L2

$$t1 = 44 \times i$$

$$a[t1] = 1$$

$$t2 = i + 1$$

if $t2 \leq 10$ goto L1
 goto t1

L2: [0][0] to [10][10]
 i = 1

Q. Write three address code for

for $(i=1; i \leq 10; i++)$

$$x = y + z \quad ; (PPQ-2023)$$

(1) $i = 1$

(2) if $i \leq 10$ goto --

(3) goto --

(4) $t1 = i + 1$

Date: _____

Day: M

(5) $x = t1$

(6) $x = goto(2)$

(7) $t2 = y + z$

(8) $x = t2$

(9) goto (4)

OR

(1) $i = 1$

L: $t1 = y + z$

$$x = t1$$

$$t2 = i + 1$$

$$i = t2$$

if $i \leq 10$ goto L1
 goto --

Date:

Q. Generate the three address code for the expression $x = A[i][j]$ for an array 10×20 . Assume $low_1 = 1$ and $low_2 = 1$.

$$low_1 = 1$$

$$low_2 = 1$$

$$n_1 = 10$$

$$n_2 = 20$$

$$A[i][j] = t_2[t_1]$$

$$t_1 = (i \times n_2 + j) \times width$$

$$t_2 = baseA - ((low_1 \times n_2) + low_2) \times width$$

$$t_1 = i \times 20$$

$$t_1 = t_1 + j$$

$$t_2 = C \quad /* \text{computation of } C = base - 84 */$$

$$t_3 = 4 \times t_1$$

$$t_4 = t_2[t_3]$$

$$x = t_4$$

Date:

Q. Generate three address code for

$$S := s + a[i][j]$$

Assume array of size 10×20 with $low_1 = 1$, $low_2 = 1$

$$t_1 = i \times 20$$

$$t_1 = t_1 + j$$

$$t_2 = C \quad /* \text{Constant } C = base - 84 */$$

$$t_3 = 4 \times t_1$$

$$t_4 = t_2[t_3]$$

$$t_5 = t_4 + t_2$$

$$S := t_5$$

Short Questions

Q. Write down equation for arrays?

The equation for 2D array

is:

$$A[i, j] = ((i * n_1) + j) * w + (base - (low_1 * n_2) + low_2)$$

Q. Why are quadruples are preferred over triples in an optimizing compiler?

In quadruple representation, the values of temporaries can be accessed using the symbol table. Hence, the quadruples are preferred over triples in optimizing compiler.

Q. List out the motivation for back patching?

⇒ Backpatching is a technique for connecting flow of control

Date:

statements into a single pass. During the code generation process, it is the action of filling in blank labels with undetermined data. Leaving the labels as empty and filling

⇒ During bottom-up passing, backpatching is utilized to generate quadruples for then later ^{during code generation} called backpatching.

⇒ In a Boolean expression suppose there are two expression E_1 & E_2 .

If the E_1 is false then there is no need to compute the rest of the expression.

But if we generate the code for the expression E_1 we must know where to jump on encountering the false E_1 expression. But this can be

accomplished by backpatching technique.

⇒ The jumping location for the

Date:

Control of flow statement is identified by backpatching.

Q. What is the intermediate code representation for the expression $a \text{ or } b$ and not c ?

The expression

$a \text{ or } b$ and not c

The address code will be:

$t_1 := a \text{ or } b$

$t_2 := \text{not } c$

$t_3 := t_1 \text{ and } t_2$

Q. What are various methods of implementing three address code statement?

⇒ The three address code can be implemented as:

1) Quadruples

2) Triples

3) Indexed triples

Date:

Q. What is the idea behind generating three address codes during compilation?

⇒ The intermediate language is an easy form of source language which can be generated efficiently by the compiler.

⇒ It leads to efficient code generation and it acts as an effective mediator between front end and back end.

⇒ Hence, it is necessary to generate three address code during compilation.

Q. What is DAG?

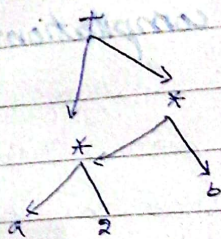
A directed Acyclic graph is an abstraction of AST that avoids the duplication.

In DAG, nodes can have multiple parents, and identical

Subtrees are shared. Such sharing makes the code more compact than conventional.

Example

$$a \times a + a \times a \times b$$



Q. Write intermediate code for procedures?

$$n = f(a[i])$$

(1) $t_1 = i \times 4$

(2) $t_2 = a[t_1]$

(3) Param t_2

(4) $t_3 = \text{call } f, 1$

(5) $n = t_3$

Q. How to write intermediate code for procedures?

$$f(a_1, a_2, a_3, a_n)$$

Param a_1

Param a_2

Date:

Day (M T W T F S)

Param a_3

Param a_n

call $f, n \leftarrow$ no of parameters
 $\rightarrow$ procedure name

Q. Write three address code for

int i, j ;

int $A[3][2]$;

for ($i=0; i<3; i++$)

{
 for ($j=0; j<2; j++$)
 {

$$A[i][j] = i + j;$$

}

}

(1) $i=0$

(2) if $i \geq 3$ goto (4)

(3) goto

(4) $j=0$

(5) if $j \geq 2$ goto

(6) $t_1 = i + 1$

(7) $i = t_1$

(8) goto (2)

- (9) $t_2 = i * 2$
 (10) $t_3 = t_2 + j$
 (11) $t_4 = t_3 * 4$
 (12) $t_5 = i + j$
 (13) $A[t_4] = t_5$
 (14) $t_6 = j + 1$
 (15) $j = t_6$
 (16) $\text{goto}(5)$

Q. Write Three address code for $A[i][j]$?

$$\begin{aligned} t_1 &= \text{addr}(A) \\ t_2 &= i * n_2 + j \\ t_3 &= t_2 * 4 \\ t_4 &= t_1[t_3] \end{aligned}$$

$n_2 = \text{total no. of columns}$
 $n_2 = \text{UB} - \text{LB} + 1$

OR

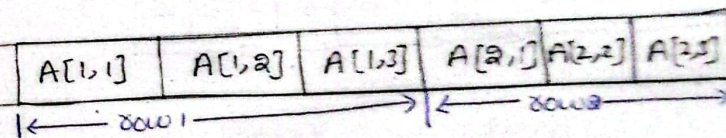
$$\begin{aligned} t_1 &= i * n_2 \\ t_2 &= t_1 * w \\ t_3 &= j * w \\ t_4 &= t_2 + t_3 \\ X &= A[t_4] \end{aligned}$$

Three address code for Array representation:

⇒ There are two methods of array representation:

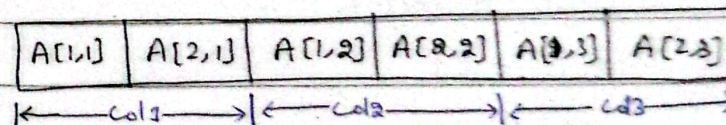
- o Row major
- o Column major

Row major representation



$$A[i][j] = BA + w * [N * (i - l_r) + (j - l_c)]$$

Column major representation



$$A[i][j] = BA + w * [(i - l_r) + m * (j - l_c)]$$

2D array

Assume $A[3][4]$

	(0)	(1)	(2)	(3)
(0)	8	6	3	2
(1)	4	5	9	1
(2)	6	3	2	4

row major

8	6	3	2	4	5	9	1	6	3	2	4
---	---	---	---	---	---	---	---	---	---	---	---

$$A[i][j] = BA + w \times (N \times (i - l_0) + (j - l_1))$$

$$= 1000 + 4 \times (4 \times (1 - 0) + (2 - 0))$$

$$= 1000 + 4(6)$$

$$= 1000 + 24$$

$$= 1024$$

Date: _____

Day: MTWTF

Column major

8	4	6	6	5	3	3	9	2	2	1	4
---	---	---	---	---	---	---	---	---	---	---	---

$$A[i][j] = BA + w \times ((i - l_0) \times N + (j - l_1))$$

$$= 1000 + 4 \times (1 \times 0 + 3 \times 0)$$

$$= 1000 + 4 \times (1 + 6)$$

$$= 1000 + 4(7)$$

$$= 1000 + 28$$

$$A[1][2] = 1028$$

if an array indexes starts from 0 then $l_0 = l_1 = 0$

$$\text{Loc}(A[i][j]) = BA + (i \times n_1 + j) \times w$$

$$= BA + i \times n_1 \times w + j \times w$$

$$= BA + i \times w_1 + j \times w$$

$$t_1 = i \times w_1$$

$$t_2 = j \times w_2$$

$$t_3 = t_1 + t_2$$

$$X = A[t_3]$$

Date: _____

Day: MTWTF

Example: ~~array~~ array
Calculate three address
code of:

declaration
statements
So, we
ignore them

```

int i
int a[10][10]
i = 0
while (i < 10)
{
    a[i][i] = 1
    i++;
}

```

- (1) $i = 0$
- (2) if ($i < 10$) goto (4)
- (3) goto (stop)
- (4) $t_1 = 44 \times i$
- (5) $a[t_1] = 1$
- (6) $t_2 = i + 1$
- (7) $i = t_2$
- (8) goto (2)
- (9) stop
- (10)

Date: _____

Day: MTWTF

$$A[i][j] = BA + w * [10 * (i - 0) + (j - 0)]$$

$$= BA + 4 * [10i + j]$$

$$A[i][i] = BA + 44i$$

1D array

Calculate three address
code of:

```

int i
int a[10]
i = 1
while (i < 10)
{
    a[i] = 0
    i = i + 1
}

```

$$A[i] = BA + w * [i - lb]$$

$$= BA + 4 * [i - 0]$$

$$= BA + 4 * i$$

So, three address code is:

- (1) $i = 1$
- (2) if ($i < 10$) goto (4)

Date:

(3) goto(3)

(4) i = i + 1

(5) a[i] = 0

(6) i = i + 1

(7) goto(3)

(8) Stop

array array, array

i = 0

for i = 0

to i = 1

do

{

a[i] = 0

i = i + 1

}

end for

stop

}

}

}

}

}

}

}

}

}

}

}

}

}

Date:

Three address code for procedures:

Procedure call:

$P(A_1, A_2, A_3, \dots, A_n)$

Param A_1

Param A_2

Param A_3

Param A_n

Call P, n $\rightarrow$ no of parameters

$\rightarrow$ Name of procedure

Example

void main()

{

int x, y;

...

Swap(&x, &y)

void swap(int *a, int *b)

{

int i;

Date: _____

Day: (M T W T F S S)

$i = *b$

$*b = *a$

$*a = i$

}

(1) call main

(2) Param &n

(3) Param &y

(4) call swap &

(5) $i = *b$

(6) $*b = *a$

(7) $*a = i$

(8) stop

$x = A[i]$

$t_1 = i * 4$

$t_2 = BA + t_1$

$= A + t_1$

OR
 $A[t_1]$

$x = A[t_1]$

3-D Array

$A[lb_1 \dots ub_1][lb_2 \dots ub_2][lb_3 \dots ub_3]$
 (i) for planes (j) for rows (k) for columns

RMO: General formula:

$$Loc(A[i][j][k]) = BA + [(i - lb_1) * n_2 + n_3 + (j - lb_2) * n_3 + (k - lb_3)] * s$$

OR

$$= BA + i * n_2 * n_3 * s + j * n_3 * s + k * s$$

OR

$$BA = (i * w + j * w_2 + k * w_3)$$

$lb_1 = lb_2 = lb_3 = 0$
 i, j, k

$$\therefore n_2 = ub_2 - lb_2 + 1, \quad \therefore n_3 = ub_3 - lb_3 + 1$$

Total no of elem

Total no of rows

Q. TAC for:

$$X = A[i][j][k]$$

$$t_1 = i * w_1$$

$$t_2 = j * w_2$$

$$t_3 = k * w_3$$

$$t_4 = t_1 + t_2$$

$$t_5 = t_4 + t_3$$

$$Q.2: A[10][20][30]$$

$$= [i * 20 * 30 + j * 30 + k] * 4$$

Q. Generate three address code for:

Switch $a+b$

```
{ case 1:  $x = x + 1$ 
  case 2:  $y = y + 2$ 
  case:  $z = z + 3$ 
  default:  $c = c + 1$ 
```

}

Ans: (1) $t_1 = a + b$

(2) if ($t_1 = 1$) goto (5)

(3) if ($t_1 = 2$) goto (7)

(4) if ($t_1 = 3$) goto (9)

5) $c = c + 1$

(6) $x = x + 1$

(8) goto (12)

(1) $y = y + 2$

(9) goto (12)

10) $z = z + 3$

11) goto (12)

~~11) $c = c + 1$~~

12) STOP